Guía de Despliegue en VPS — Ubuntu (Lumina CRM / Bhealth Insurance)

Documentación completa para desplegar y mantener la aplicación en un **VPS Ubuntu 24.04** (probado en Hostinger). Cubre instalación inicial, configuración, seguridad, actualizaciones y resolución de problemas.

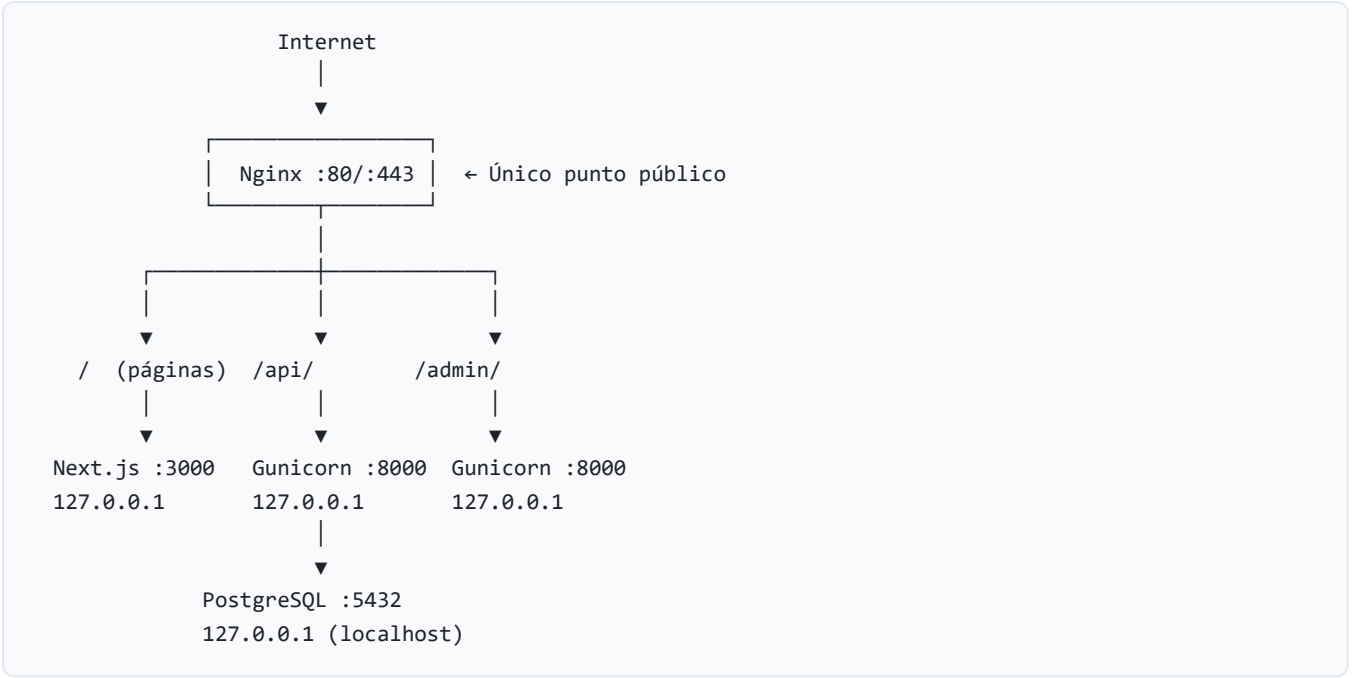
Documentación relacionada:

Archivo	Contenido
`API_CRUD_DETAILED.md`	API backend
`FRONTEND_DOC.md`	Frontend Next.js
`deploy/README.md`	Referencia rápida de plantillas

Tabla de Contenidos

1. Arquitectura en Producción
2. Requisitos
3. Preparación del Servidor
4. PostgreSQL
5. Clonar el Repositorio
6. Configurar Backend (.env)
7. Entorno Python y Backend
8. Build del Frontend
9. Servicios systemd
10. Nginx (Reverse Proxy)
11. Firewall (UFW)
12. Usuario Admin CRM
13. Deploy Key de GitHub
14. Verificación Post-Deploy
15. Actualizar la Aplicación
16. Fail2ban (SSH)
17. Backups de Base de Datos
18. Dominio y HTTPS
19. Mantenimiento
20. Solución de Problemas
21. Referencia Rápida de Comandos

Arquitectura en Producción



Componente	Puerto	Usuario sistema	Acceso público
Nginx	80, 443	root	Sí
Next.js	3000	www-data	**No** (solo localhost)
Gunicorn (Django)	8000	www-data	**No** (solo localhost)
PostgreSQL	5432	postgres	**No** (solo localhost)
SSH	22	—	Sí (restringido)

Rutas asumidas en esta guía:

Recurso	Ruta
Código	`/var/www/Bhealth-Insurance`
venv Python	`/var/www/Bhealth-Insurance/venv`
Backend `.env`	`/var/www/Bhealth-Insurance/backend/.env`
Frontend build	`/var/www/Bhealth-Insurance/frontend/.next`
Media uploads	`/var/www/Bhealth-Insurance/backend/media`
Estáticos Django	`/var/www/Bhealth-Insurance/backend/staticfiles`
Backups DB	`/var/backups/lumina`

Reemplaza `2.25.68.83` por la IP pública de tu VPS en todos los ejemplos.

Requisitos

VPS mínimo recomendado

Recurso	Mínimo	Recomendado
CPU	2 vCPU	4 vCPU
RAM	4 GB	8 GB
Disco	40 GB SSD	80 GB SSD
SO	Ubuntu 22.04/24.04 LTS	Ubuntu 24.04 LTS

Software necesario

- Ubuntu 24.04 LTS (Hostinger u otro proveedor)
- Acceso SSH (usuario `deploy` con llave SSH)
- Dominio opcional (para HTTPS con Let's Encrypt)
- Repositorio Git: `github.com:Dzherrom/Bhealth-Insurance.git`

En tu PC local

- Git
- Cliente SSH
- (Opcional) Acceso a GitHub para configurar Deploy Key

Preparación del Servidor

Conéctate como `root` o con un usuario con `sudo` la primera vez.

1. Actualizar el sistema

```
sudo apt update && sudo apt upgrade -y
sudo apt install -y git curl wget build-essential \
  python3 python3-pip python3-venv python3-dev \
  postgresql postgresql-contrib \
  nginx ufw fail2ban \
  nodejs npm
```

> **Node.js:** En Ubuntu 24.04 el `nodejs` del repositorio puede ser antiguo. Si `npm run build` falla, instala Node 20 LTS:

```
curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -
sudo apt install -y nodejs
node -v   # debe ser v20+
```

2. Crear usuario `deploy`

```
sudo adduser deploy
sudo usermod -aG sudo deploy
```

3. Configurar SSH con llave (desde tu PC)

En tu PC (PowerShell):

```
ssh-keygen -t ed25519 -C "vps-deploy" -f $env:USERPROFILE\.ssh\id_ed25519_vps
type $env:USERPROFILE\.ssh\id_ed25519_vps.pub
```

En el VPS (como root):

```
mkdir -p /home/deploy/.ssh
nano /home/deploy/.ssh/authorized_keys
# Pegar la línea pública de tu PC
chmod 700 /home/deploy/.ssh
chmod 600 /home/deploy/.ssh/authorized_keys
chown -R deploy:deploy /home/deploy/.ssh
```

Probar desde tu PC:

```
ssh deploy@TU_IP_VPS
```

4. (Opcional) Deshabilitar login root por contraseña

```
sudo nano /etc/ssh/sshd_config
# PermitRootLogin prohibit-password  (o no)
# PasswordAuthentication no
sudo systemctl restart sshd
```

5. Crear directorio de la aplicación

```
sudo mkdir -p /var/www
sudo chown deploy:deploy /var/www
```

PostgreSQL

Crear base de datos y usuario

Como root o con sudo:

```
sudo -u postgres psql
```

```
CREATE USER lumina_administrator WITH PASSWORD 'TU_PASSWORD_SEGURO_AQUI';
CREATE DATABASE lumina_db OWNER lumina_administrator;
GRANT ALL PRIVILEGES ON DATABASE lumina_db TO lumina_administrator;
\q
```

URL-encode de la contraseña

Si la contraseña tiene caracteres especiales (`+` , `/` , `=`), codifícala:

```
python3 -c "import urllib.parse; print(urllib.parse.quote('TU_PASSWORD', safe=''))"
```

Usa el resultado en `DATABASE_URL` .

Verificar conexión

```
psql "postgresql://lumina_administrator:PASSWORD@localhost:5432/lumina_db" -c "SELECT 1;"
```

Clonar el Repositorio

Como usuario `deploy` :

```
cd /var/www
git clone git@github.com:Dzherrom/Bhealth-Insurance.git
cd Bhealth-Insurance
```

Si aún no tienes Deploy Key (ver sección 13), usa HTTPS temporalmente:

```
git clone https://github.com/Dzherrom/Bhealth-Insurance.git
```

Configurar safe.directory (si git pull falla como root)

```
git config --global --add safe.directory /var/www/Bhealth-Insurance
```

Configurar Backend (.env)

Nunca commitear `backend/.env` — contiene secretos.

```
cd /var/www/Bhealth-Insurance
cp backend/.env.example backend/.env
nano backend/.env
```

Plantilla para producción (VPS)

```
SECRET_KEY=GENERA_UNA_CLAVE_LARGA_AQUI
DEBUG=False

DATABASE_URL=postgresql://lumina_administrator:PASSWORD_URL_ENCODED@localhost:5432/lumina_db

BACKEND_PUBLIC_URL=http://2.25.68.83
FRONTEND_ORIGINS=http://2.25.68.83
ALLOWED_HOSTS=2.25.68.83,srv1781611.hstgr.cloud

DJANGO_SUPERUSER_USERNAME=lumina-admin
DJANGO_SUPERUSER_EMAIL=admin@lumina.local
DJANGO_SUPERUSER_PASSWORD=TuContraseñaSegura123!
```

Generar `SECRET_KEY`:

```
openssl rand -base64 64
```

Permisos del archivo:

```
sudo chown deploy:www-data backend/.env
sudo chmod 640 backend/.env
```

Entorno Python y Backend

```
cd /var/www/Bhealth-Insurance

# Crear venv
python3 -m venv venv
source venv/bin/activate

# Instalar dependencias
pip install --upgrade pip
pip install -r requirements.txt

# Migraciones y estáticos
cd backend
python manage.py migrate
python manage.py collectstatic --noinput

# Crear admin CRM
python manage.py ensure_crm_admin

# Verificar
python manage.py check
deactivate
```

Permisos media y staticfiles

```
sudo mkdir -p /var/www/Bhealth-Insurance/backend/media
sudo mkdir -p /var/www/Bhealth-Insurance/backend/staticfiles
sudo chown -R www-data:www-data /var/www/Bhealth-Insurance/backend/media
sudo chown -R www-data:www-data /var/www/Bhealth-Insurance/backend/staticfiles
```

Build del Frontend

Importante: el build se hace como `deploy`, **antes** de pasar ownership a `www-data`.

```
cd /var/www/Bhealth-Insurance/frontend

# Crear .env.production ANTES del build
cat > .env.production << 'EOF'
NEXT_PUBLIC_SITE_URL=http://2.25.68.83
EOF

npm ci
npm run build
```

Sin `NEXT_PUBLIC_SITE_URL`, el middleware puede redirigir a `localhost:3000` en producción.

Después del build:

```
sudo chown -R www-data:www-data /var/www/Bhealth-Insurance/frontend
```

Servicios systemd

Backend (Gunicorn)

```
sudo cp /var/www/Bhealth-Insurance/deploy/systemd/lumina-backend.service /etc/systemd/system/
sudo systemctl daemon-reload
sudo systemctl enable lumina-backend
sudo systemctl start lumina-backend
sudo systemctl status lumina-backend
```

El servicio ejecuta:

```
gunicorn config.wsgi:application --bind 127.0.0.1:8000 --workers 2
```

Usuario: `www-data` · Lee `backend/.env` vía `EnvironmentFile`.

Frontend (Next.js)

```
sudo cp /var/www/Bhealth-Insurance/deploy/systemd/lumina-frontend.service /etc/systemd/system/
sudo systemctl daemon-reload
sudo systemctl enable lumina-frontend
sudo systemctl start lumina-frontend
sudo systemctl status lumina-frontend
```

Next.js escucha solo en `127.0.0.1:3000` (no expuesto a internet).

Ver logs

```
sudo journalctl -u lumina-backend -n 50 --no-pager
sudo journalctl -u lumina-frontend -n 50 --no-pager
sudo journalctl -u lumina-backend -f    # seguir en vivo
```

Nginx (Reverse Proxy)

```
# Editar server_name si tu IP/hostname es diferente
sudo nano /var/www/Bhealth-Insurance/deploy/nginx/lumina-insurance.conf

sudo cp /var/www/Bhealth-Insurance/deploy/nginx/lumina-insurance.conf \
  /etc/nginx/sites-available/lumina-insurance
sudo ln -sf /etc/nginx/sites-available/lumina-insurance /etc/nginx/sites-enabled/
sudo rm -f /etc/nginx/sites-enabled/default

sudo nginx -t
sudo systemctl reload nginx
```

Rutas que Nginx expone

Ruta pública	Destino
`/`	Next.js `127.0.0.1:3000`
`/api/`	Gunicorn `127.0.0.1:8000`
`/admin/`	Gunicorn `127.0.0.1:8000`
`/static/`	Gunicorn (Whitenoise)
`/media/`	Archivos en `backend/media/`

Comprobar

```
curl -I http://127.0.0.1:8000/api/
curl -I http://127.0.0.1:3000/
curl -I http://2.25.68.83/
curl -I http://2.25.68.83/auth/login
```

Firewall (UFW)

Solo deben ser públicos **22** (SSH), **80** y **443** (HTTP/HTTPS).

```
sudo ufw default deny incoming
sudo ufw default allow outgoing
sudo ufw allow OpenSSH
sudo ufw allow 'Nginx Full'
sudo ufw deny 3000/tcp
sudo ufw deny 8000/tcp
sudo ufw enable
sudo ufw status verbose
```

También revisa el **firewall del panel Hostinger** (hPanel) y deja solo puertos 22, 80, 443.

Usuario Admin CRM

Tras configurar `DJANGO_SUPERUSER_*` en `backend/.env`:

```
cd /var/www/Bhealth-Insurance/backend
source ../venv/bin/activate
python manage.py ensure_crm_admin
```

Acceso	URL
CRM (empleados)	`http://2.25.68.83/auth/login`
Django Admin	`http://2.25.68.83/admin/`

Resetear contraseña:

```
python manage.py ensure_crm_admin --update-password
```

Deploy Key de GitHub

Para que el VPS pueda hacer `git pull` sin contraseña:

1. Generar llave en el VPS (como deploy)

```
ssh-keygen -t ed25519 -C "vps-deploy-bhealth-github" -f ~/.ssh/id_ed25519_github -N ""
cat ~/.ssh/id_ed25519_github.pub
```

2. Añadir en GitHub

1. Ir a: `https://github.com/Dzherrom/Bhealth-Insurance/settings/keys`
2. **Add deploy key**
3. Title: `vps-deploy-bhealth`
4. Pegar la llave pública
5. Marcar **Allow write access** (si necesitas `git push` desde el VPS)

3. Configurar SSH para GitHub

```
cat >> ~/.ssh/config << 'EOF'

Host github.com
  HostName github.com
  User git
  IdentityFile ~/.ssh/id_ed25519_github
  IdentitiesOnly yes
EOF
chmod 600 ~/.ssh/config
```

4. Probar

```
ssh -T git@github.com
cd /var/www/Bhealth-Insurance && git pull
```

Problema: `backend/.env` bloquea git pull

Si Git dice que `backend/.env` sería sobrescrito:

```
cp backend/.env /tmp/lumina-backend.env.bak
mv backend/.env /tmp/   # o git stash
git pull
mv /tmp/lumina-backend.env.bak backend/.env
sudo chown deploy:www-data backend/.env
sudo chmod 640 backend/.env
```

Verificación Post-Deploy

```
chmod +x /var/www/Bhealth-Insurance/deploy/scripts/verify-deploy.sh
bash /var/www/Bhealth-Insurance/deploy/scripts/verify-deploy.sh
```

Checklist manual

#	Verificación	Comando / Acción
1	Servicios activos	`sudo systemctl status lumina-backend lumina-frontend nginx postgresql`
2	Puertos internos	`ss -tlnp | grep -E '3000|8000'` → `127.0.0.1`
3	API responde	`curl -I http://2.25.68.83/api/`
4	Admin Django	`curl -I http://2.25.68.83/admin/` (no 502)
5	Login CRM	Abrir `/auth/login` en navegador
6	Login funciona	DevTools → POST a `/api/login/` (no Render)
7	Puertos cerrados	`:3000` y `:8000` no accesibles desde fuera
8	CRM redirect	`/crm/dashboard` no redirige a `localhost:3000`

Actualizar la Aplicación

Tras cada `git push` desde tu PC:

```
cd /var/www/Bhealth-Insurance

# IMPORTANTE: respaldar .env antes de pull
cp backend/.env /tmp/lumina-backend.env.bak

chmod +x deploy/scripts/update-app.sh
bash deploy/scripts/update-app.sh

# Restaurar .env si el script lo sobrescribió
cp /tmp/lumina-backend.env.bak backend/.env
sudo chown deploy:www-data backend/.env
sudo chmod 640 backend/.env

bash deploy/scripts/verify-deploy.sh
```

Qué hace `update-app.sh`

1. `chown deploy:deploy` del proyecto (permite git pull y npm build)
2. `git pull`
3. `pip install -r requirements.txt`
4. `migrate` + `collectstatic` + `ensure_crm_admin`
5. `npm ci` + `npm run build`
6. Restaura permisos (`www-data` en frontend, media, staticfiles)
7. Reinicia `lumina-backend`, `lumina-frontend`, reload nginx

Actualizar solo frontend

```
sudo chown -R deploy:deploy /var/www/Bhealth-Insurance/frontend
cd /var/www/Bhealth-Insurance/frontend
git pull
npm ci && npm run build
sudo chown -R www-data:www-data /var/www/Bhealth-Insurance/frontend
sudo systemctl restart lumina-frontend
```

Actualizar solo backend

```
cd /var/www/Bhealth-Insurance
source venv/bin/activate
pip install -r requirements.txt
cd backend
python manage.py migrate
python manage.py collectstatic --noinput
deactivate
sudo systemctl restart lumina-backend
```

Fail2ban (SSH)

Protege contra fuerza bruta en SSH (3 intentos fallidos → ban 1 hora).

```
sudo apt install -y fail2ban
sudo cp /var/www/Bhealth-Insurance/deploy/fail2ban/jail.local /etc/fail2ban/jail.local
sudo systemctl enable fail2ban
sudo systemctl restart fail2ban
sudo fail2ban-client status sshd
```

Desbloquear tu IP

Si quedas bloqueado (desde consola Hostinger o como root):

```
sudo fail2ban-client set sshd unbanip TU_IP_PUBLICA
```

Backups de Base de Datos

Configurar

```
sudo mkdir -p /var/backups/lumina
sudo chown deploy:deploy /var/backups/lumina
chmod +x /var/www/Bhealth-Insurance/deploy/scripts/backup-db.sh
```

Backup manual

```
bash /var/www/Bhealth-Insurance/deploy/scripts/backup-db.sh
ls -lh /var/backups/lumina/
```

Genera: `lumina_db_YYYYMMDD_HHMMSS.sql.gz` (retención 14 días por defecto).

Cron diario (como deploy)

```
crontab -e
```

```
0 3 * * * /var/www/Bhealth-Insurance/deploy/scripts/backup-db.sh >> /var/backups/lumina/backup.log 2>&
```

Restaurar backup

```
# Leer DATABASE_URL de backend/.env primero
gunzip -c /var/backups/lumina/lumina_db_YYYYMMDD_HHMMSS.sql.gz | \
  psql "postgresql://lumina_administrator:PASSWORD@localhost:5432/lumina_db"
```

Dominio y HTTPS

Cuando tengas un dominio apuntando al VPS:

1. DNS

Registro **A** → IP del VPS (`2.25.68.83`).

2. Actualizar `backend/.env`

```
BACKEND_PUBLIC_URL=https://tu-dominio.com
FRONTEND_ORIGINS=https://tu-dominio.com
ALLOWED_HOSTS=tu-dominio.com,2.25.68.83
```

3. Actualizar `frontend/.env.production`

```
NEXT_PUBLIC_SITE_URL=https://tu-dominio.com
```

4. Certificado SSL (Let's Encrypt)

```
sudo apt install -y certbot python3-certbot-nginx
sudo certbot --nginx -d tu-dominio.com
```

5. Rebuild y restart

```
bash /var/www/Bhealth-Insurance/deploy/scripts/update-app.sh
```

6. Verificar

- `https://tu-dominio.com/auth/login`
- Cookies seguras activas (`DEBUG=False` + HTTPS en Django)

Mantenimiento

Tarea	Frecuencia	Comando
Verificar deploy	Tras cada actualización	`bash deploy/scripts/verify-deploy.sh`
Logs backend	Semanal	`journalctl -u lumina-backend -n 100`
Backup DB	Diario (cron)	`backup-db.sh`
Actualizaciones SO	Mensual	`sudo apt update && sudo apt upgrade -y`
Snapshot VPS	Antes de cambios grandes	Panel Hostinger hPanel
Rotar secretos	Si hubo filtración	`SECRET_KEY`, passwords DB, admin

Actualizaciones automáticas del SO

```
sudo apt install -y unattended-upgrades
sudo dpkg-reconfigure -plow unattended-upgrades
```

Solución de Problemas

502 Bad Gateway en `/admin/` o `/api/`

Causa: Gunicorn no está corriendo en `127.0.0.1:8000`.

```
sudo systemctl status lumina-backend
sudo journalctl -u lumina-backend -n 50 --no-pager
sudo ss -tlnp | grep 8000
sudo systemctl restart lumina-backend
```

Failed to fetch en login (frontend)

Causa	Solución
Backend caído	Reiniciar `lumina-backend`
Frontend apunta a Render/URL incorrecta	Rebuild con `NEXT_PUBLIC_SITE_URL` correcto
CORS / ALLOWED_HOSTS	Verificar `FRONTEND_ORIGINS` y `ALLOWED_HOSTS` en `.env`
Permisos `.env`	`chown deploy:www-data` + `chmod 640`

Redirect a `localhost:3000`

1. Crear `frontend/.env.production` con `NEXT_PUBLIC_SITE_URL=http://TU_IP`
2. `npm run build`
3. Verificar `proxy_redirect` en nginx (ya incluido en plantilla)
4. Reiniciar frontend y nginx

`EACCES` en `npm ci` o `npm run build`

El directorio `frontend/` pertenece a `www-data`. Recuperar ownership:

```
sudo chown -R deploy:deploy /var/www/Bhealth-Insurance/frontend
# hacer build...
sudo chown -R www-data:www-data /var/www/Bhealth-Insurance/frontend
```

`git pull` — Permission denied (publickey)

Configurar Deploy Key de GitHub.

`git pull` — untracked files would be overwritten

```
sudo chown -R deploy:deploy /var/www/Bhealth-Insurance
cp backend/.env /tmp/backup.env
# mover o eliminar archivos conflictivos
git pull
cp /tmp/backup.env backend/.env
```

venv corrupto (`python: command not found`)

```
cd /var/www/Bhealth-Insurance
rm -rf venv
python3 -m venv venv
source venv/bin/activate
pip install -r requirements.txt
sudo systemctl restart lumina-backend
```

PostgreSQL — connection refused

```
sudo systemctl status postgresql
sudo systemctl start postgresql
# Verificar DATABASE_URL en backend/.env
```

Fail2ban bloqueó tu IP

```
sudo fail2ban-client set sshd unbanip TU_IP
```

Frontend no arranca — falta `.next`

```
cd /var/www/Bhealth-Insurance/frontend
sudo chown -R deploy:deploy .
npm ci && npm run build
sudo chown -R www-data:www-data .
sudo systemctl restart lumina-frontend
```

Referencia Rápida de Comandos

```
# Estado general
sudo systemctl status lumina-backend lumina-frontend nginx postgresql

# Reiniciar todo
sudo systemctl restart lumina-backend lumina-frontend
sudo systemctl reload nginx

# Logs
sudo journalctl -u lumina-backend -n 50 --no-pager
sudo journalctl -u lumina-frontend -n 50 --no-pager

# Deploy completo
cd /var/www/Bhealth-Insurance && bash deploy/scripts/update-app.sh

# Verificar
bash deploy/scripts/verify-deploy.sh

# Backup DB
bash deploy/scripts/backup-db.sh

# Django shell
cd /var/www/Bhealth-Insurance/backend && source ../venv/bin/activate && python manage.py shell

# Crear/resetear admin
python manage.py ensure_crm_admin --update-password
```

URLs de acceso (producción)

Servicio	URL
CRM Login	`http://TU_IP/auth/login`
CRM Dashboard	`http://TU_IP/crm/dashboard`
Portal Cliente	`http://TU_IP/client`
API	`http://TU_IP/api/`
Django Admin	`http://TU_IP/admin/`

Resumen del Proceso (Primera Instalación)

```
1.  Preparar Ubuntu (apt update, paquetes)
2.  Crear usuario deploy + SSH key
3.  PostgreSQL (usuario + DB)
4.  git clone → /var/www/Bhealth-Insurance
5.  backend/.env (SECRET_KEY, DATABASE_URL, ALLOWED_HOSTS…)
6.  python3 -m venv + pip install + migrate + collectstatic
7.  frontend/.env.production + npm ci + npm run build
8.  systemd: lumina-backend + lumina-frontend
9.  nginx: lumina-insurance.conf
10. ufw: solo 22, 80, 443
11. ensure_crm_admin
12. Deploy Key GitHub (para futuros git pull)
13. verify-deploy.sh
14. fail2ban + backups cron
15. (Opcional) dominio + certbot HTTPS
```

Última actualización: Julio 2026
VPS de referencia: Ubuntu 24.04 · Hostinger · IP `2.25.68.83`
Servicios: `lumina-backend`, `lumina-frontend`, `nginx`, `postgresql`